

Conversion a CRUD y conexion al frontend

Que se construyo sobre la base de datos, como se verifico, y que quedo deliberadamente fuera. Todas las cifras de este informe se midieron consultando la API y la base en ejecucion — ninguna se tomo de la documentacion.

38/39	211	103	190
recursos con CRUD completo	endpoints	pruebas de API	pruebas de frontend

Resumen en una linea. El CRUD paso de **0 recursos completos de 26 a 38 de 39**, la API de 91 a 211 endpoints, y las pantallas dejaron de mostrarse vacias: obligaciones pasó de “no hay obligaciones” a listar las 5 reales. En el camino aparecieron **tres fallas de seguridad** y **cinco defectos** que no estaban en el plan.

Contenido

SECCION	QUE CUBRE
1 · Punto de partida	Que habia antes, medido
2 · La capa de datos	SQLAlchemy, borrado logico, aislamiento
3 · El CRUD	Las 52 tablas y que se hizo con cada grupo
4 · Hallazgos	Lo que aparecio al construir, con su gravedad
5 · Conexion al frontend	Por que las pantallas se veian vacias
6 · Pruebas	La evidencia, ejecutada contra el sistema real
7 · Lo que queda	Pendientes, con su motivo

1 · Punto de partida

El estado inicial era peor que “incompleto”. Medido sobre el contrato OpenAPI antes de empezar:

OPERACION	RECURSOS QUE LA TENIAN
Crear	24 de 26
Listar	22 de 26
Leer uno	12 de 26
Actualizar	15 de 26
Borrar	0 de 26

Ningun recurso tenia borrado, y el problema iba mas hondo que un endpoint faltante. `CRUDBase.remove()` hacia borrado **físico** mientras el esquema estaba diseñado para borrado **lógico**. Al mismo tiempo, las lecturas no filtraban las filas borradas. Las dos mitades estaban mal y se tapaban entre sí, porque ningun router exponia el borrado: exponerlo directo lo habria hecho nacer roto.

Ademas, de las 52 tablas de la base, **21 no tenian ninguna API** — el 40 % del modelo no se podia tocar desde la aplicacion. Y un endpoint devolvía 500: `POST /support/tickets` no generaba su numero de ticket. Era el unico roto de los 91, y justamente el que sostiene el flujo del Cliente Invitado, cuyo unico proposito es crear solicitudes.

2 · La capa de datos

Como esta armada

`routers/` reciben y responden · `crud/` un `CRUDBase` generico por modelo · `models/` `SQLAlchemy 2.0`.

Las tablas **no** se crean desde `SQLAlchemy`: no existe `create_all()` en el proyecto. El esquema vive en SQL y se aplica por migracion, porque incluye Row Level Security, triggers y restricciones que un ORM no genera. `SQLAlchemy` se usa para consultar, no para definir.

Los 26 recursos eran **instancias directas de `CRUDBase` sin sobreescribir nada**. Eso significa que arreglarlo alcanzo a todos desde un solo archivo — y que un error ahi se propaga igual de rapido.

El borrado paso a ser logico

`remove()` marca la fecha de borrado y las lecturas excluyen lo borrado desde un unico punto. Dos decisiones que vale explicar:

La marca la pone la base, no Python. Asi la hora es la del servidor de datos y es comparable con las de creacion y modificacion, que ya se generan ahi.

`get()` **paso de `db.get()` a `select()`**. No fue estilo: `db.get()` resuelve por clave primaria y por el mapa de identidad, asi que nunca pasa por un `WHERE` donde filtrar. No habia forma de agregarle la condicion.

Borrar dos veces no es un error ni mueve la fecha original. Desde afuera, borrar algo ya borrado y borrar algo que nunca existio son el mismo hecho: los dos responden 404.

3 · El CRUD sobre las 52 tablas

“50 tablas, hacerlas rapido” fue el encargo. Son 52, pero **no todas son recursos de API**: las tablas de union se administran desde su padre, las bitacoras las escribe el sistema, y los catalogos de referencia se consultan. Contarlas todas infla la estimacion.

GRUPO	N	QUE SE HIZO
Recursos de negocio	27	CRUD completo, verificado
Tablas anidadas	5	Expuestas bajo su recurso padre
Plantillas globales	2	Lectura abierta, escritura solo Admin Global
Catalogo normativo	9	Lectura abierta, escritura solo Admin Global
Bloqueadas por RBAC	5	Sin aprobar la especificacion
Inmutables por diseno	2	La base revoca UPDATE y DELETE
Otras (union, bitacoras)	2	No llevan CRUD propio

Las tres que necesitaron diseno propio

Claves compuestas. Participantes de auditoria, operadores de equipo y procesos por planta se identifican por dos columnas, no por un id. CRUDBase lanza un error explicito sobre ellas — a proposito, para que nadie las conecte por accidente — y se exponen anidadas bajo su padre. No son simples enlaces: llevan atributos propios, como el rol en la auditoria o la certificacion del operador.

Plantillas y catalogo. No llevan identificador de empresa: lo que se crea ahi lo ven **todas**. No bastaba con escribir el router, habia que decidir quien puede escribir. Leer, cualquier usuario autenticado; escribir, solo Admin Global.

Contratos. El esquema aceptaba la empresa gestora desde el cuerpo del request, asi que una empresa podia declararse gestora de otra. Ahora la fija el servidor con la empresa de la sesion. **Lo que no se resuelve** es el consentimiento de la contraparte: nombrar a otra empresa como cliente sigue siendo unilateral. Hoy el dano esta acotado porque el contrato solo lo ve quien lo creo, pero deja de estarlo el dia que algo conceda permisos en base a un contrato.

4 · Lo que aparecio al construir

Ninguno estaba en el plan. Todos aparecieron al ejecutar, no al leer.

Tres fallas de seguridad

1 · El listado de empresas estaba abierto sin autenticacion

Devolvía la cartera completa de clientes —RUT, razón social y giro— a cualquiera que alcanzara la API, y permitía crear y editar empresas. No fue descuido: es la única tabla de negocio sin identificador de empresa, así que Row Level Security no la cubre y la dependencia habitual no le servía. Al no haber una que encajara, quedó sin ninguna.

2 · El aislamiento entre empresas se perdía en cada confirmación de transacción

La protección dependía de una instrucción que muere con la transacción. Medido sobre una base con 6 usuarios repartidos en 2 empresas:

CONEXION	OLVIDANDO TODO	TRAS CONFIRMAR
Dueno de la base (antes)	6 = las dos empresas	6
Rol de aplicacion (ahora)	0	0

Y **ya estaba ocurriendo**: al serializar la respuesta de cada creación o edición se disparaba una consulta sin protección. La API ahora se conecta con un rol que no puede saltarse Row Level Security, así que el modo de fallo se invirtió: de “*ve todo*” a “*no ve nada*”.

3 · Las claves foraneas no pasan por Row Level Security

Una restricción de integridad solo exige que la fila exista; **no mira a que empresa pertenece**. Una edición con la planta de otra empresa pasaba sin problema. El dano no es la fila incoherente: es un **oráculo de existencia**, porque quien prueba identificadores al azar distingue “no existe” de “existe pero es de otro”. Por eso ambos casos responden con el mismo error.

Cinco defectos

DEFECTO	CONSECUENCIA
Numero de ticket sin generar	El unico endpoint roto de los 91, y el que sostiene el acceso del Cliente Invitado
Sin manejo de errores de integridad	Un valor invalido devolvía 500 en toda la API: quien integra no sabía que corregir
Claves unicas no parciales sobre el borrado	Volver a agregar algo que se quito daba 500. En tablas de union eso es la operacion normal
Anidar la ruta no ata el hijo al padre	Un recurso se devolvía aunque perteneciera a otro padre: la jerarquia de la URL era decorativa
Sin proteccion contra ciclos en jerarquias	Un ciclo no rompe la insercion: cuelga a quien recorra el arbol despues

5 · Conexion al frontend

El bloqueo no eran los adaptadores de datos. Ocho de los doce ya consultaban la API. El problema es que **21 pantallas sacaban su lista de plantas de datos de ejemplo fijos**, con identificadores inventados, mientras la API devuelve identificadores reales. Los datos llegaban bien y el filtro por planta no cruzaba con ninguno: las pantallas se veian vacias **con los datos ahi**.

Tres errores de mapeo

La misma clase de defecto que el proyecto ya tenia documentada: el modelo esta escrito dos veces —una en el backend y otra en el frontend— y se desincronizo.

LEIA	DEBIA LEER	CONSECUENCIA
due_date	due_at	Toda obligacion mostraba hoy como vencimiento
assigned_user_id	owner_user_id	Ningun responsable, nunca
4 estados	8 estados	Traducia dos que no existen . Los 6 reales caian al valor por defecto: una obligacion rechazada se mostraba vigente

Un hallazgo de modelado

La pantalla de “departamentos” se conecto a **procesos**, no a departamentos. Los nombres divergieron, pero los datos no dejan lugar a dudas: tipo, entradas, salidas y responsable viven en procesos. El propio esquema compartido lo explica — *“se modela como proceso porque es lo que ISO 9001 pide identificar”*.

Los enlaces directos por URL rebotaban

Escribir una direccion a mano terminaba siempre en el tablero; por el menu funcionaba. La causa estaba en **24 pantallas**, todas con el mismo control heredado de la epoca de los datos de ejemplo: preguntaban por una clave de almacenamiento local que con el proveedor de identidad real nunca existe.

El arreglo no fue en las pantallas sino en el origen: **“no hay sesion” y “todavia no se” eran indistinguibles**. Ahora la sesion expone si esta resolviendose, y las 24 pantallas preguntan por las dos cosas. El defecto estaba latente tambien en el modo sin proveedor — solo que ahi el control lo tapaba por accidente.

Resultado, verificado en pantalla

PANTALLA	ANTES	AHORA
Obligaciones	<i>“No hay obligaciones”</i>	5 obligaciones con sistema, periodo, estado y vencimiento correctos
Matriz Legal	<i>“No hay normas”</i>	3 normas con su cobertura
Filtro de plantas	Plantas inventadas	Planta Calama · Faena Antofagasta · Oficina Santiago
Cabecera	“Ambienta”	“Minera Andes SpA”
URL directa	Rebotaba al tablero	Se queda donde se pidio

6 · Pruebas

Todo lo de esta seccion se ejecuto contra el sistema corriendo, con una sesion real, no con simulaciones. Los datos de prueba se eliminaron al terminar.

Ciclo completo por recurso

RECURSO	CREAR	LISTAR	LEER	EDITAR	BORRAR	RELEER
departments	201	200	200	200	204	404
processes	201	200	200	200	204	404
facilities	201	200	200	200	204	404
contracts	201	200	200	200	204	404
declarations	201	200	200	200	204	404
documents	201	200	200	200	204	404
iso14001/aspects	201	200	200	200	204	404
support/tickets	201	200	200	200	204	404
integrations	201	200	200	200	204	404
audits/{id}/items	201	200	200	200	204	404

El **404 final** es la prueba del borrado logico: la fila queda marcada en la base y desaparece de la API. **10 de 10 ciclos completos.**

Aislamiento entre empresas

PRUEBA	RESULTADO	SIGNIFICA
Peticion sin token	401	No hay acceso anonimo a datos de negocio
Token de una empresa + cabecera de otra	Solo la propia	La cabecera no cruza empresas
Listar empresas	1	Cada quien ve la suya, no la cartera
Leer una empresa ajena	404	No confirma que exista
Editar una empresa ajena	404	Igual: no se distingue de inexistente
Crear empresa sin ser Admin Global	403	Alta reservada a la administracion

Permisos sobre datos compartidos

PRUEBA	RESULTADO
Escribir una plantilla sin ser Admin Global	403
Escribir el catalogo sin ser Admin Global	403
Leer plantillas con sesion normal	200

Validaciones e integridad

PRUEBA	RESULTADO
Un registro como su propio padre	422
Referencia a algo que no existe en la empresa	422
Hijo pedido bajo el padre equivocado	404
Codigo duplicado	201 el primero, 409 el segundo
Valor fuera de los permitidos	422, nombrando la restriccion
Reinstalar un vinculo que se habia quitado	201

Las pruebas se comprobaron rompiendo a proposito lo que dicen proteger. Al quitar el filtro de borrado fallan 3; al volver al borrado fisico fallan 2; al quitar la validacion de referencias fallan 2. No son pruebas vacias.

Suites automatizadas

SUITE	ANTES	AHORA	ESTADO
Pruebas de API	57	103	Verde
Pruebas de frontend	190	190	Verde
Comprobaciones del esquema	9	9	Verde
Integracion continua	1 de 3 rojo	3 de 3	Verde

La integracion continua tambien se arreglo: creaba el esquema **despues** de correr las pruebas y solo aplicaba 2 de los 7 archivos de migracion. Daba verde sobre un esquema que no es el que corre en ningun lado.

7 · Lo que queda fuera, y por que

Cada exclusion tiene un motivo declarado, y una prueba automatizada falla si alguien agrega un recurso a medias sin explicar por que. Esa prueba ya sirvió dos veces: encontro operaciones faltantes que no estaban en la lista de nadie.

QUE	MOTIVO	TIPO
Registro de auditoria e historial de estados	La base revoca las operaciones de modificacion y borrado. Es un requisito no funcional: el registro es inmutable	Imposible
Roles y permisos (5 tablas)	Pertenecen a una especificacion sin aprobar. Exponerlas ahora anticiparia decisiones de diseno que no se han tomado	Bloqueado
Borrado de empresas	Marcar una empresa como dada de baja no impide entrar a sus usuarios: seria una baja que miente	Sin resolver
Consentimiento en contratos	Nombrar a otra empresa como cliente sigue siendo unilateral. Falta el flujo de aceptacion	Sin resolver

Riesgos conocidos

RIESGO	DETALLE
Claves unicas por codigo	No excluyen las filas borradas. Borrar un departamento y volver a crearlo con el mismo codigo daria un error de servidor. Se resolvió en las tablas de union, donde es la operacion normal; en el resto sigue abierto
Autoria sin registrar	Las columnas de quien creo y quien modifiko existen pero nunca se llenan. Se puede cambiar la estructura organizacional sin dejar rastro de quien lo hizo
Consultas por instalacion	La matriz legal pide las asignaciones una vez por planta. Son 3 o 4 para una empresa tipica; con decenas de faenas hace falta un endpoint que las devuelva juntas
Sin control de permisos por rol	Cualquier usuario autenticado de una empresa puede escribir sobre sus datos. La barrera de empresa funciona; la de rol no existe todavia

Estado de la base de datos

52	38	156	9/9
tablas	con aislamiento	claves foraneas	comprobaciones

PostgreSQL 16. La base se recrea desde cero con un comando y queda verificada: se comprobo aplicando los siete archivos de migracion sobre una base vacia, en el mismo orden que usa la integracion continua.

Pendiente del plan original. El encargo tenia dos días: el primero para el CRUD y el segundo para llevarlo al frontend. El primero esta completo y verificado. Del segundo se resolvió el bloqueo estructural —las pantallas ya leen datos reales— pero **quedan formularios y acciones de escritura sin conectar**: hoy crear o editar desde la interfaz actualiza el estado local, no siempre la base.